STATOIL / C-CORE ICEBERG CLASSIFIER

Complete Beginner-Friendly Project Guide

Ship vs. Iceberg Classification using 5 Deep Learning Models

Google Colab • Python • TensorFlow/Keras • Kaggle Dataset

1. Purpose of This Guide

This document is written for students who are new to this dataset and to image classification. The goal is not only to provide code, but to explain what the data means, why each processing step is necessary, what each model is doing, how the models should be compared, and how to produce the final Kaggle submission.

The project will compare five models: a Custom CNN, ResNet50, EfficientNetB0, MobileNetV2, and DenseNet121. The experiment should be conducted using the same train/validation split and comparable training settings so that the model comparison is meaningful.

2. What Students Should Understand by the End

- What SAR (Synthetic Aperture Radar) imagery is at a high level.
- What band_1 and band_2 represent and why both are useful.
- Why each band contains 5,625 values and how those values become a 75×75 image.
- What the fields id, band_1, band_2, inc_angle, and is_iceberg mean.
- How to explore and visualize the dataset before training.
- How to prepare the SAR data for CNNs.
- What train/validation splitting and stratification mean.
- Why data augmentation is useful and why it must be conservative for SAR imagery.
- The difference between a CNN trained from scratch and transfer learning.
- How to train, evaluate, compare, and select among five models.
- Why log loss is especially important for this competition.
- How to generate predictions and a Kaggle submission file.

3. The Complete Project at a Glance

1. Set up Google Colab and install/import required libraries.
2. Download the Kaggle competition data.
3. Load train.json and test.json.
4. Understand the dataset schema.
5. Explore dimensions, data types, missing values, and class balance.
6. Visualize band_1 and band_2 for ships and icebergs.
7. Reshape each flattened band from 5,625 values to 75×75.
8. Construct the model input representation.
9. Normalize/preprocess the image data.
10. Create a stratified training/validation split.
11. Define controlled data augmentation.
12. Train the Custom CNN baseline.
13. Train ResNet50 using transfer learning.
14. Train EfficientNetB0 using transfer learning.
15. Train MobileNetV2 using transfer learning.
16. Train DenseNet121 using transfer learning.
17. Evaluate all five models with common metrics.
18. Plot learning curves, confusion matrices, and ROC curves.
19. Compare models in one results table.
20. Select the best model using validation performance, with emphasis on log loss.
21. Retrain/fine-tune the selected model according to the final experimental protocol.
22. Predict probabilities for test.json.

23. Create submission.csv and submit it to Kaggle.

4. Understand the Dataset Before Writing a Model

The competition data is provided in JSON format. Each training record describes one 75×75 SAR image chip. The dataset fields include an image id, two flattened radar bands, an incidence angle, and the training label. The documented schema states that band_1 and band_2 each contain 75×75 = 5,625 values.

Training record conceptually:

```
{
  "id": "example_id",
  "band_1": [5625 values],
  "band_2": [5625 values],
  "inc_angle": 43.9,
  "is_iceberg": 1
}
```

Important interpretation:

Field	Meaning	Student takeaway
id	Unique identifier for the image chip.	Keep it unchanged for the final submission.
band_1	First SAR polarization measurement; documented as HH.	5,625 numbers → reshape to 75×75.
band_2	Second SAR polarization measurement; documented as HV.	5,625 numbers → reshape to 75×75.
inc_angle	Incidence angle associated with image acquisition.	Inspect missing values; initially keep the image-only experiment simple.
is_iceberg	Training target: 1 = iceberg, 0 = ship.	This is the label the model learns to predict.

4.1 What are the two bands?

Students should not think of band_1 and band_2 as two different objects. They are two radar measurements of the same target. The documented interpretation is dual-polarization SAR: HH (horizontal transmit / horizontal receive) and HV (horizontal transmit / vertical receive). Ships and icebergs can produce different scattering patterns in these measurements.

A useful mental model is: one ocean target → two complementary radar measurements → one classification decision.

4.2 Why 5,625 values?

Each band is a flattened 75×75 image:

```
75 × 75 = 5,625

band_1: [x1, x2, ..., x5625]  → reshape(75, 75)

band_2: [x1, x2, ..., x5625]  → reshape(75, 75)
```

Flattening means the two-dimensional image was stored as one long list. Reshaping restores the spatial arrangement needed by a CNN.

5. Stage 1 — Set Up Google Colab

Recommended environment: Google Colab with a GPU runtime.

24. Open a new Google Colab notebook.
25. Enable a GPU through Runtime → Change runtime type → GPU.
26. Import Python libraries for data handling, numerical computation, visualization, and deep learning.
27. Confirm that the GPU is visible.
28. Create a clean project folder structure.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

Do not continue until the notebook runs without import/runtime errors.

6. Stage 2 — Obtain the Kaggle Data

Download the competition data from the Kaggle competition page. The working files are train.json and test.json. Keep the original files unchanged and work on copies/loaded data in the notebook.

- Verify that train.json exists.
- Verify that test.json exists.
- Do not manually edit the JSON files.
- Record the exact dataset location used in the notebook.

7. Stage 3 — Load and Inspect the JSON

```
train = pd.read_json("train.json")
test = pd.read_json("test.json")

print(train.shape)
print(test.shape)
print(train.columns)
print(test.columns)
train.head()
```

Students should answer these questions before moving on:

- How many rows are in train and test?
- What columns are present in each file?
- Which column is the target?
- Are band_1 and band_2 stored as lists?
- Does inc_angle contain missing/non-numeric entries?

8. Stage 4 — Exploratory Data Analysis (EDA)

8.1 Check data types and missing values

```
train.info()
train.isna().sum()
train["inc_angle"].value_counts(dropna=False).head(20)
```

The incidence-angle field has been reported in example analyses to contain 'na' entries. Do not blindly replace missing values without documenting the decision. For the first image-only model comparison, the students can keep inc_angle out of the model input and treat it as an inspected auxiliary field.

8.2 Check class balance

```
train["is_iceberg"].value_counts()
train["is_iceberg"].value_counts(normalize=True)
```

Visualize the class distribution with a bar chart. Record the counts in the project report.

9. Stage 5 — Visualize the SAR Images

Do not train a model before looking at the data. Visualization is used to verify that the reshape operation is correct and to build intuition about what the network receives.

29. Select a few ship samples and a few iceberg samples.
30. Convert their band lists to 75×75 arrays.
31. Display band_1 and band_2 separately.
32. Add the true label to each visualization.
33. Check whether the two bands show different intensity/spatial patterns.
34. Look for suspicious samples or obvious preprocessing problems.

```
idx = 0
b1 = np.array(train.loc[idx, "band_1"], dtype=np.float32).reshape(75, 75)
b2 = np.array(train.loc[idx, "band_2"], dtype=np.float32).reshape(75, 75)

plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.imshow(b1, cmap="gray")
plt.title("Band 1 (HH)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(b2, cmap="gray")
plt.title("Band 2 (HV)")
plt.axis("off")
plt.show()
```

10. Stage 6 — Convert Flattened Bands to Images

```
X_band_1 = np.array(
   [np.array(band, dtype=np.float32).reshape(75, 75)
    for band in train["band_1"]]
)

X_band_2 = np.array(
   [np.array(band, dtype=np.float32).reshape(75, 75)
    for band in train["band_2"]]
)

print(X_band_1.shape)
print(X_band_2.shape)
```

Expected conceptual shape:

```
(number_of_samples, 75, 75)
```

At this point, each training example has two 75×75 matrices. Nothing has been classified yet; this is only data representation.

11. Stage 7 — Build the CNN Input

The original data is naturally dual-channel (HH and HV). For this project, the main five-model comparison will use a three-channel representation so that standard ImageNet-pretrained architectures can be used conveniently.

```
band_3 = (X_band_1 + X_band_2) / 2.0

X = np.stack([X_band_1, X_band_2, band_3], axis=-1)

print(X.shape)
```

Expected shape:

```
(number_of_samples, 75, 75, 3)
```

Interpretation: channel 1 = HH, channel 2 = HV, channel 3 = average of HH and HV. This representation is a project design choice; students should state it explicitly in the report.

12. Stage 8 — Normalize the Image Data

The band values are SAR/backscatter values rather than ordinary RGB values. They should therefore be transformed into a numerically suitable range before model training. A published competition analysis reports that the supplied band values were in decibels and explored normalization before deep-learning training.

For a clean educational experiment, choose one normalization method, document it, and apply it consistently. Importantly, any statistics learned from the training data must not leak information from the validation or test sets.

Recommended simple protocol for the first experiment:

35. Split the original labeled data into training and validation sets first.
36. Compute any learned normalization statistics using the training portion only.
37. Apply the same transformation to validation and test data.
38. Keep the transformation identical for all five models.

If using a fixed per-image or global scaling formula, document the exact formula in the notebook and report.

13. Stage 9 — Create the Training and Validation Sets

The labels are available only in train.json. Do not use test.json to tune the models. Create one fixed validation split and reuse it for all five models.

```
from sklearn.model_selection import train_test_split

y = train["is_iceberg"].astype(int).values

X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
```

```
    stratify=y
)
```

Why stratify? Because it keeps the approximate ship/iceberg class proportion similar in training and validation. Why fixed random_state? Because every model must be evaluated on the same validation examples for a fair comparison.

14. Stage 10 — Data Augmentation

The training set is relatively small. Controlled augmentation can expose the model to slightly varied versions of training images. Only apply augmentation to the training data; never augment validation or test data for evaluation.

- Possible: horizontal flip.
- Possible: vertical flip.
- Possible: small rotations.
- Possible: small translations if justified.
- Avoid aggressive transformations that could change the physical meaning of the SAR target.

Keep augmentation identical across all five model experiments.

15. Stage 11 — Define a Fair Five-Model Experiment

Use a common experimental protocol:

Experimental item	Recommended rule
Training/validation split	Same fixed stratified split for all models.
Input	Same 75×75×3 representation.
Augmentation	Same augmentation policy.
Target	is_iceberg: 0/1.
Primary competition metric	Log loss.
Additional metrics	Accuracy, precision, recall, F1, ROC-AUC.
Callbacks	Early stopping and best-checkpoint saving.
Randomness	Use documented seeds where practical.
Model selection	Choose based on validation performance, not test predictions.

16. Stage 12 — Model 1: Custom CNN

Purpose: establish a transparent baseline that students can understand. It is trained from scratch and shows what a small CNN can achieve before using large pretrained architectures.

Suggested structure:

```
Input (75×75×3)
→ Conv2D
→ Batch Normalization
→ ReLU
→ Average/Max Pooling
→ Conv2D
→ Batch Normalization
→ ReLU
→ Pooling
→ Conv2D
→ Global Average Pooling
→ Dense
```

→ Dropout
→ Sigmoid output

The exact number of filters, dropout, learning rate, and epochs should be recorded as experiment parameters.

17. Stage 13 — Model 2: ResNet50

ResNet50 is a deep residual CNN. We will use transfer learning: load pretrained convolutional features, replace the final classification head, train the head first, and optionally fine-tune selected upper layers.

Input (75×75×3)
→ ResNet50 backbone
→ Global Average Pooling
→ Dropout
→ Dense(1, sigmoid)

Important: pretrained ImageNet features are not SAR-specific. This is an experimental transfer-learning baseline, not a guarantee of superiority.

18. Stage 14 — Model 3: EfficientNetB0

EfficientNetB0 provides a compact modern CNN architecture. Use the same transfer-learning and fine-tuning protocol used for ResNet50.

Input → EfficientNetB0 backbone → Global Average Pooling → Dropout → Sigmoid

19. Stage 15 — Model 4: MobileNetV2

MobileNetV2 is designed to be computationally efficient. It provides a useful comparison between classification performance and model size.

Input → MobileNetV2 backbone → Global Average Pooling → Dropout → Sigmoid

20. Stage 16 — Model 5: DenseNet121

DenseNet121 connects layers through dense feature reuse. Use the same transfer-learning protocol and compare it using the same validation set.

Input → DenseNet121 backbone → Global Average Pooling → Dropout → Sigmoid

21. Stage 17 — Training Protocol for Models 2–5

39. Load the pretrained backbone without its original classification head.
40. Attach a new binary classification head.
41. Freeze the backbone initially.
42. Train only the new head for a short initial phase.
43. Monitor validation loss.
44. Optionally unfreeze the final portion of the backbone.
45. Fine-tune with a much smaller learning rate.
46. Restore the best validation checkpoint.
47. Record training time, parameter counts, and best validation metrics.

Do not unfreeze the entire network immediately. With a small dataset, aggressive fine-tuning can overfit.

22. Stage 18 — Evaluate Every Model

The model outputs a probability p = P(iceberg). For classification metrics, a threshold such as 0.5 can be used. For log loss, retain the probability itself rather than converting it to 0/1.

Metric	Meaning
Accuracy	Fraction of validation examples classified correctly at the chosen threshold.
Precision	Among predicted icebergs, fraction that are actually icebergs.
Recall	Among actual icebergs, fraction detected by the model.
F1-score	Harmonic mean of precision and recall.
ROC-AUC	Measures ranking/separation quality across thresholds.
Log loss	Probability-sensitive loss; especially important because it was the competition metric.

Binary log loss concept:

```
LogLoss = - mean[
   y * log(p) + (1-y) * log(1-p)
]
```

Lower log loss is better. A highly confident wrong prediction is penalized strongly, which is why probability calibration matters.

23. Stage 19 — Analyze Learning Behaviour

23.1 Training vs validation loss

Plot both curves. If training loss continues to decrease while validation loss starts increasing, the model is likely overfitting.

23.2 Training vs validation accuracy

Use this plot together with loss; accuracy alone can hide poor probability quality.

23.3 Confusion matrix

```
                Predicted
              Ship  Iceberg
Actual Ship     TN     FP
Actual Iceberg  FN     TP
```

23.4 ROC curves

Plot all five ROC curves together using the same validation labels.

24. Stage 20 — Build the Final Comparison Table

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Log Loss	Params	Time
Custom CNN	—	—	—	—	—	—	—	—
ResNet50	—	—	—	—	—	—	—	—
EfficientNetB0	—	—	—	—	—	—	—	—
MobileNetV2	—	—	—	—	—	—	—	—
DenseNet121	—	—	—	—	—	—	—	—

Fill this table only after evaluating all models on the same validation set. Do not change the validation split from one model to another.

25. Stage 21 — Select the Best Model

Do not choose a model solely because it has the highest accuracy. For this competition, log loss deserves special attention. Also consider F1/ROC-AUC, overfitting behaviour, parameter count, and training/inference cost.

- Primary criterion: validation log loss.
- Secondary evidence: ROC-AUC, F1, accuracy, precision, recall.
- Check learning curves for overfitting.
- Check whether the model is stable across repeated seeds if time permits.
- Consider computational efficiency when two models perform similarly.

26. Stage 22 — Prepare test.json

Only after the model selection is complete should the final model be used to generate predictions for test.json. Apply exactly the same reshape, channel construction, and normalization pipeline used for training/validation.

```
X_test_band_1 = np.array(
    [np.array(band, dtype=np.float32).reshape(75, 75)
     for band in test["band_1"]]
)

X_test_band_2 = np.array(
    [np.array(band, dtype=np.float32).reshape(75, 75)
     for band in test["band_2"]]
)

X_test = np.stack([
    X_test_band_1,
    X_test_band_2,
    (X_test_band_1 + X_test_band_2) / 2.0
], axis=-1)
```

27. Stage 23 — Generate submission.csv

The submission should contain the test id and the predicted probability of iceberg.

```
pred = best_model.predict(X_test).ravel()

submission = pd.DataFrame({
    "id": test["id"],
    "is_iceberg": pred
})

submission.to_csv("submission.csv", index=False)
```

Before submitting, verify:

- The number of rows equals the number of test examples.
- The id column matches test.json ids.
- is_iceberg contains probabilities, not hard 0/1 labels.
- There are no NaN or infinite predictions.
- The CSV has exactly the required column names.

28. Reproducibility Checklist

- Record Python/TensorFlow versions.
- Record whether GPU was used.
- Fix random seeds where practical.
- Keep one fixed stratified validation split.
- Record model architecture and hyperparameters.
- Save the best model checkpoint.
- Record training time.
- Record all validation metrics.
- Save plots and the final comparison table.
- Save submission.csv and note the Kaggle submission score.

29. Recommended Google Colab Notebook Structure

48. 01. Title and Project Objective
49. 02. Imports and Environment Check
50. 03. Download/Locate Kaggle Data
51. 04. Load train.json and test.json
52. 05. Dataset Schema and EDA
53. 06. Missing Values and Class Distribution
54. 07. Visualize Band 1 and Band 2
55. 08. Reshape 5,625 Values → 75×75
56. 09. Create 3-Channel Input
57. 10. Train/Validation Split
58. 11. Normalization
59. 12. Data Augmentation
60. 13. Model 1 — Custom CNN
61. 14. Model 2 — ResNet50
62. 15. Model 3 — EfficientNetB0
63. 16. Model 4 — MobileNetV2
64. 17. Model 5 — DenseNet121
65. 18. Evaluate All Models
66. 19. Learning Curves
67. 20. Confusion Matrices
68. 21. ROC Curves
69. 22. Final Comparison Table
70. 23. Best Model Selection
71. 24. Test Prediction
72. 25. Create submission.csv
73. 26. Final Conclusions

30. Common Mistakes Students Should Avoid

Mistake	Correct practice
Treating band_1 and band_2 as unrelated images.	They are two measurements of the same target.
Forgetting to reshape 5,625 values.	Each band must be restored to 75×75 spatial form.
Using the test set to choose hyperparameters.	Test labels are hidden; use validation data for development.
Changing the validation split for each model.	Use one fixed split for fair comparison.
Applying augmentation to validation/test data.	Augmentation belongs only in the training pipeline.

Reporting only accuracy.	Include log loss and other metrics.
Converting probabilities to 0/1 for Kaggle submission.	The competition uses probability-sensitive log loss.
Aggressive fine-tuning of pretrained networks.	Small datasets can overfit quickly.
Forgetting to save the best checkpoint.	The final epoch is not necessarily the best epoch.
Not recording preprocessing choices.	A result cannot be reproduced without the preprocessing protocol.

31. Final Deliverables for the Student Team

- One clean Google Colab notebook containing the complete pipeline.
- Five trained model experiments.
- Training/validation loss and accuracy plots for each model.
- Confusion matrix for each model.
- ROC curve comparison.
- Final model comparison table.
- Saved best-model checkpoint.
- submission.csv.
- A short conclusion explaining which model performed best and why.
- A record of the exact preprocessing, split, augmentation, and hyperparameters used.

32. Suggested Final Project Report Structure

74. Introduction and problem statement.
75. Dataset description.
76. SAR bands and feature representation.
77. Data preprocessing.
78. Experimental setup.
79. Five model architectures.
80. Training procedure.
81. Evaluation metrics.
82. Results and model comparison.
83. Error analysis.
84. Best-model selection.
85. Kaggle submission result.
86. Limitations and future work.
87. Conclusion.